PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 06 -

KUBERNETES VIA TERRAFORM: USANDO OS PROVIDERS KUBERNETES E KUBECTL

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 11
O QUE VOCÊ VIU NESTA AULA? 12
REFERÊNCIAS 13

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Kubernetes via Terraform: Usando os Providers Kubernetes e Kubectl

O QUE VEM POR AÍ?

Gerenciar um cluster Kubernetes costuma ser um desafio de organização e consistência. Muitas vezes, times diferentes acabam aplicando manifestos YAML manualmente, cada um do seu jeito, o que gera divergências e dificulta o controle de versões. Agora, vamos explorar como o **Terraform** pode ser usado não apenas para provisionar infraestrutura, mas também para **gerenciar recursos dentro do Kubernetes**, trazendo para esse ecossistema toda a disciplina da **Infraestrutura como Código**.

Nesta aula, você vai conhecer os **providers Kubernetes e Kubectl**, entendendo as diferenças e quando usar cada um deles. Veremos como criar e aplicar recursos — como *Deployments*, *Services*, *ConfigMaps* e *Secrets* — diretamente via Terraform, de forma declarativa e rastreável. Também vamos discutir casos práticos, como o deploy de uma aplicação simples, e refletir sobre as limitações dessa abordagem, avaliando quando faz sentido complementar o Terraform com outras ferramentas como **Helm** ou **ArgoCD**.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

Nesta etapa prática, vamos demonstrar como o Terraform pode ser usado para interagir diretamente com clusters Kubernetes já provisionados. A ideia não é apenas provisionar infraestrutura na nuvem, mas também gerenciar os recursos internos do Kubernetes com o mesmo fluxo de versionamento e automação que aplicamos em IaC até aqui.

Nos vídeos, o time vai configurar e utilizar dois providers diferentes: o **Kubernetes** e o **Kubectl**. O primeiro permite criar recursos de forma declarativa diretamente no cluster (como *Deployments*, *Services* e *ConfigMaps*), enquanto o segundo possibilita aplicar manifestos YAML já existentes, como se estivéssemos rodando `kubectl apply`, mas dentro do fluxo controlado pelo Terraform. Essa combinação mostra a flexibilidade de integrar práticas tradicionais do ecossistema Kubernetes com a disciplina do IaC.

A prática vai incluir a criação de um namespace dedicado, o deploy de uma aplicação simples (um servidor Nginx) e a gestão de objetos de configuração, como ConfigMaps e Secrets. Com isso, será possível observar como o Terraform se conecta ao cluster, aplica os recursos, mantém o estado versionado e garante reprodutibilidade. A proposta é compreender não só o funcionamento técnico, mas também como essa abordagem ajuda a padronizar o ciclo de vida das aplicações em Kubernetes.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

O Kubernetes é poderoso, mas também complexo. Ele não é apenas um orquestrador de containers, é um verdadeiro **sistema operacional distribuído** para a nuvem. E como todo sistema complexo, a forma como organizamos e controlamos seus recursos faz toda a diferença na confiabilidade e na produtividade do time.

Muitas empresas ainda vivem um cenário onde cada desenvolvedor mantém seus próprios manifests YAML no repositório da aplicação, aplicando manualmente com ***kubectl apply***. Isso funciona no início, mas à medida que o número de serviços cresce, a situação fica insustentável: podem surgir **diferenças entre ambientes**, configurações duplicadas e até **erros de segurança**.

É nesse contexto que o Terraform brilha. Ele permite trazer os recursos do Kubernetes para dentro do mesmo ciclo de **Infraestrutura como Código (IaC)** que você já usou para criar clusters, VPCs e bancos de dados. Assim, tudo fica versionado, revisado por pares e com histórico de mudanças.

Providers Kubernetes e Kubectl

Para integrar Terraform e Kubernetes, existem dois caminhos principais: o **Kubernetes Provider** e o **Kubectl Provider**.

- O **Kubernetes Provider** funciona como uma tradução de objetos Kubernetes para HCL. Você escreve um Deployment, um Service ou um ConfigMap diretamente em blocos Terraform, sem precisar de YAML. Isso é ótimo para quem quer padronização completamente.
- O **Kubectl Provider** é mais flexível. Ele aceita manifests YAML já existentes e os aplica como se fosse um ***kubectl apply.*** Assim, você pode aproveitar arquivos já existentes, como um chart Helm exportado, sem ter que reescrever tudo.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Exemplo — Kubernetes Provider

```
provider "kubernetes" {
  config_path = "~/.kube/config"
}

resource "kubernetes_namespace" "demo" {
  metadata {
    name = "demo"
  }
}

resource "kubernetes_deployment" "nginx" {
  metadata {
    name      = "nginx-deploy"
    namespace = kubernetes_namespace.demo.metadata[0].name
  }
  spec {
    replicas = 2
    selector {
      match_labels = { app = "nginx" }
    }
    template {
      metadata {
        labels = { app = "nginx" }
      }
      spec {
        container {
          name  = "nginx"
          image = "nginx:1.25"
          port { container_port = 80 }
        }
      }
    }
  }
}
```

Esse exemplo cria um namespace **demo** e um Deployment de Nginx. Perceba que tudo está em HCL, integrado ao estado do Terraform. Se o time aumentar réplicas de 2 para 3, a mudança é rastreada e aplicada de forma consistente.

Exemplo — Kubectl Provider

```
provider "kubectl" {
  config_path = "~/.kube/config"
}

resource "kubectl_manifest" "nginx_svc" {
  yaml_body = <<YAML
apiVersion: v1
kind: Service
metadata:
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
    name: nginx-service
    namespace: demo
  spec:
    selector:
      app: nginx
    ports:
    - port: 80
      targetPort: 80
  YAML
  }
```

Aqui, em vez de escrever **HCL**, você cola o **YAML** de um Service pronto. Isso é prático quando o time já possui muitos ***manifests*** ou quando um fornecedor disponibiliza arquivos **YAML**.

Para saber mais: a documentação oficial do Kubernetes Provider não é apenas uma lista de recursos, mas também uma forma de entender a cobertura e limitações do Terraform dentro do ecossistema Kubernetes. Ela mostra que nem todos os objetos suportados pelo Kubernetes têm blocos equivalentes em HCL. Isso significa que, em alguns casos, você pode precisar complementar sua abordagem com o Kubectl Provider ou até usando Helm Charts.

Ao estudar essa documentação, você descobre que é possível criar desde os recursos mais básicos, como Namespaces e Pods, até objetos mais avançados, como Ingress e políticas de rede. Você encontrará exatamente o que o Terraform consegue gerenciar e até onde ele pode ir sem precisar de ferramentas adicionais. Isso ajuda a definir **estratégias de adoção híbridas** no seu time.

ConfigMaps, Secrets e boas práticas

Um dos maiores ganhos de usar Terraform com Kubernetes está na padronização de **ConfigMaps** e **Secrets**, que são frequentemente esquecidos ou mal versionados.

ConfigMap com Terraform

```
resource "kubernetes_config_map" "app_config" {
  metadata {
    name      = "app-config"
    namespace = "demo"
  }
  data = {
    APP_ENV = "staging"
    LOG_LVL = "info"
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
    }
  }
```

Secret com Terraform

```
resource "kubernetes_secret" "db_secret" {
  metadata {
    name      = "db-credentials"
    namespace = "demo"
  }

  data = {
    username = base64encode("admin")
    password = base64encode("supersecret")
  }
}
```

Evite versionar Secrets diretamente no Git! O exemplo acima é apenas didático. Em produção, integre com **Vault**, **AWS Secrets Manager** ou operadores de secrets no Kubernetes.

Para saber mais: integrar o Terraform com o HashiCorp Vault vai muito além de proteger Secrets. O Vault pode gerar credenciais dinâmicas sob demanda, por exemplo, criar um usuário de banco de dados que expira após algumas horas. Assim, você não apenas evita armazenar senhas no Git, mas também reduz a superfície de ataque caso uma credencial seja exposta.

Estudando o Vault Provider, você percebe como isso se conecta ao ciclo IaC: em vez de codificar um segredo, o Terraform consulta o Vault em tempo de execução e injeta a informação de forma transitória. Essa abordagem muda a cultura de segurança, trazendo práticas de **segurança zero-trust** e eliminando dependência de credenciais fixas.

Deploy de aplicações mais complexas

O Terraform também pode orquestrar aplicações mais robustas. Imagine instalar um **Ingress Controller** ou aplicar políticas de RBAC:

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

RBAC com Kubernetes Provider

```
resource "kubernetes_role" "read_pods" {
  metadata {
    name      = "read-pods"
    namespace = "demo"
  }
  rule {
    api_groups = [""]
    resources  = ["pods"]
    verbs      = ["get", "list"]
  }
}

resource "kubernetes_role_binding" "read_pods_binding" {
  metadata {
    name      = "read-pods-binding"
    namespace = "demo"
  }

  role_ref {
    api_group = "rbac.authorization.k8s.io"
    kind      = "Role"
    name      = kubernetes_role.read_pods.metadata[0].name
  }
  subject {
    kind      = "User"
    name      = "demo-user"
    api_group = "rbac.authorization.k8s.io"
  }
}
```

Esse exemplo mostra como definir permissões diretamente no Terraform. Assim, fica claro quem pode fazer o quê em cada namespace, evitando permissões manuais inconsistentes.

Para saber mais: aprender RBAC no Kubernetes é essencial para ambientes com múltiplos times. A documentação oficial explica como permissões são aplicadas de forma hierárquica: Roles e RoleBindings controlam acessos em namespaces, enquanto ClusterRoles e ClusterRoleBindings aplicam políticas no cluster inteiro.

Estudando esse tema, você descobre padrões comuns, como separar permissões por papéis (ex.: “developer”, “admin”, “qa”) e aplicar princípio do menor privilégio, garantindo que cada usuário tenha apenas os acessos que precisa. Essa

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

prática evita erros acidentais, como exclusão de recursos críticos, e melhora a governança em times grandes.

Quando usar Helm, ArgoCD ou FluxCD

Nem sempre o Terraform é a melhor ferramenta para gerenciar aplicações dentro do Kubernetes. Projetos grandes geralmente usam uma **abordagem híbrida**:

- **Terraform:** cria a base como clusters, namespaces, quotas, ingress controllers.
- **Helm:** instala aplicações complexas como Prometheus, Grafana ou Istio.
- **ArgoCD**: mantém aplicações sincronizadas com repositórios Git (GitOps).

O Helm Provider leva o Terraform além da criação de objetos individuais. Com ele, você pode instalar aplicações completas como Prometheus, Grafana ou Istio com poucas linhas de código. Isso acontece porque os charts já empacotam centenas de linhas de YAML, cobrindo desde Deployments até ConfigMaps e RBAC.

Ao explorar isso, você percebe como o Terraform pode integrar infraestrutura e aplicações. Ele garante consistência no provisionamento dos charts e facilita rollback, já que cada versão instalada fica registrada no estado. Isso dá ao time confiança para automatizar a entrega de ferramentas de observabilidade ou de service mesh sem complicar o processo.

O ArgoCD introduz uma mentalidade diferente: em vez de você publicar configurações para o cluster, o próprio cluster “puxa” as mudanças declaradas em um repositório Git. Isso garante que o estado do cluster esteja sempre sincronizado com a versão declarada, reduzindo o risco de drift.

Ao estudar ArgoCD, você vai além da prática manual de aplicar manifests. Aprende sobre **GitOps**, um modelo em que o Git se torna a fonte única de verdade, e o cluster se autogerencia. Isso traz benefícios como auditoria integrada (cada mudança tem histórico no Git), facilidade de rollback e alinhamento com práticas modernas de DevOps.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

O uso de Kubernetes em larga escala exige que times combinem diferentes ferramentas para alcançar governança, consistência e velocidade. Um exemplo prático desse movimento está no projeto documentado no artigo *"Build a Real-World DevOps Pipeline with GitHub Actions, Terraform, EKS and Argo CD"*.

Nesse estudo, é mostrado como unir **Terraform** para provisionar infraestrutura (neste caso, clusters EKS na AWS), **GitHub Actions** para automação de pipelines de CI/CD e **ArgoCD** para implementar o modelo de **GitOps** dentro do Kubernetes. O fluxo funciona da seguinte forma: o Terraform prepara toda a base necessária — redes, subnets, EKS e permissões — e, em seguida, o ArgoCD assume o papel de sincronizar as aplicações dentro do cluster a partir de repositórios Git.

O ponto forte desse case é mostrar como as ferramentas se complementam. O Terraform garante que a infraestrutura seja criada de forma declarativa e rastreável, enquanto o ArgoCD mantém os manifestos de aplicações sempre atualizados e sincronizados, eliminando drift. O GitHub Actions integra tudo em um pipeline automatizado, permitindo que cada alteração em código ou configuração passe por etapas de validação antes de chegar ao ambiente final.

Essa combinação de IaC, automação e GitOps está se tornando um padrão de mercado, e estudar esse tipo de projeto ajuda a entender como práticas modernas de DevOps e Kubernetes se unem para construir pipelines escaláveis e confiáveis.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Kubernetes via Terraform: Usando os Providers Kubernetes e Kubectl

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu que o Terraform pode ir além da criação de infraestrutura na nuvem e também ser aplicado diretamente dentro do **Kubernetes**, usando os providers **Kubernetes** e **Kubectl**. Viu como cada um deles funciona: o primeiro para declarar recursos em HCL, garantindo padronização e versionamento, e o segundo para reutilizar manifests YAML já existentes.

Exploramos exemplos práticos de como criar **namespaces, deployments, services, configmaps e secrets**, trazendo esses objetos para dentro do ciclo de IaC. Também discutimos práticas de segurança, como o uso de gerenciadores externos para secrets e o princípio do **menor privilégio (least privilege)** em permissões RBAC.

Por fim, entendemos que, embora o Terraform seja poderoso, ele não substitui todas as ferramentas do ecossistema Kubernetes. O caminho mais eficaz é adotar uma abordagem **híbrida**, usando Terraform para a base do cluster e recursos fixos, e complementando com soluções como **Helm** e **ArgoCD** para aplicações mais dinâmicas e complexas.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

HASHICORP. **Kubernetes Provider**. Terraform Registry, 2025. Disponível em: https://registry.terraform.io/providers/hashicorp/kubernetes/latest/docs. Acesso em: 21 out. 2025.

HASHICORP. **Kubectl Provider**. Terraform Registry, 2025. Disponível em: https://registry.terraform.io/providers/gavinbunney/kubectl/latest/docs. Acesso em: 21 out. 2025.

HASHICORP. **Helm Provider**. Terraform Registry, 2025. Disponível em: https://registry.terraform.io/providers/hashicorp/helm/latest/docs. Acesso em: 21 out. 2025.

HASHICORP. **Vault Provider**. Terraform Registry, 2025. Disponível em: https://registry.terraform.io/providers/hashicorp/vault/latest/docs. Acesso em: 21 out. 2025.

KUBERNETES. **Using RBAC Authorization**. Kubernetes Documentation, 2025. Disponível em: https://kubernetes.io/docs/reference/access-authn-authz/rbac/. Acesso em: 21 out. 2025.

NEAMUL, Kabir Emon. **Build a Real-World DevOps Pipeline with GitHub Actions, Terraform, EKS and Argo CD — Step by Step**. Medium, 2025. Disponível em: https://medium.com/@neamulkabiremon/build-a-real-world-devops-pipeline-with-github-actions-terraform-eks-and-argo-cd-step-by-step-c568f1efd29e. Acesso em: 21 out. 2025.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

terraform. argocd. kubernetes, automação.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com